GUIA VISUAL 037

COMPILANDO DO CÓDIGO-FONTE

Seis meses depois, ninguém sabe que aquele binário existe — nem de onde veio, nem como sai.

037
GUIAS LINUX

1 FONTE
`tar xzf · git clone`
Verifique antes: `sha256sum -c` prova *integridade*; só `gpg --verify` (ou tag assinada) prova *origem*.

→

2 CONFIGURADO
`./configure`
Não compila nada: **descobre o que existe nesta máquina** e escreve o Makefile. Falhou aqui = falta um pacote `-dev`.

→

3 COMPILADO
`make -j"$(nproc)"`
A etapa demorada, ainda dentro do diretório de fonte. Quebrou com `-j` e passou sem? Makefile com dependência mal declarada.

→

4 INSTALADO
`sudo make install`
Só **copia arquivos**. É a única etapa que pede `sudo` — e a única que o sistema **não registra**.

SISTEMA	CONFIGURAR	COMPILAR	INSTALAR
Autotools	`./configure --prefix=/usr/local`	`make -j$(nproc)`	`sudo make install`
CMake	`cmake -B build -DCMAKE_BUILD_TYPE=Release`	`cmake --build build -j$(nproc)`	`sudo cmake --install build`
Meson	`meson setup build --prefix=/usr/local`	`ninja -C build`	`sudo ninja -C build install`

O PREFIXO NÃO É ESCOLHA ESTÉTICA

/usr
TERRITÓRIO DO DPKG

Todo arquivo aqui pertence a algum pacote, e o `dpkg -S` sabe a qual.

O gerenciador pode **sobrescrever ou remover** qualquer coisa daqui numa atualização.

/usr/local
TERRITÓRIO DO ADMINISTRADOR

O FHS reserva esta árvore para software instalado localmente. **Nenhum pacote Debian escreve aqui.**

É o padrão do `configure` justamente por isso — e o `PATH` típico já o põe **antes** de `/usr/bin`.

Por que `--prefix=/usr` morde: seu binário fica no mesmo caminho de um do repositório. Se o pacote oficial for instalado depois, ele **sobrescreve o seu sem aviso**; se for removido, leva junto arquivos que você achava seus. Terceira opção, sem `sudo` algum: `--prefix=$HOME/.local`, válida só para o seu usuário. E antes de deixar o `make install` agir, veja o que ele faria: `make install DESTDIR=/tmp/preview` + `find /tmp/preview -type f` é o *dry-run* que o `make` não tem.

`MAKE INSTALL` CRU CRIA SOFTWARE INVISÍVEL — TRÊS SAÍDAS

MAKE UNINSTALL — FRÁGIL
Só funciona **do mesmo diretório de fonte**, configurado com o mesmo prefixo. Apagou o diretório, perdeu a desinstalação — e nem todo projeto implementa o alvo.

CHECKINSTALL — O ATALHO BOM
Intercepta o `make install`, observa os arquivos criados e empacota tudo num `.deb`. Passa a aparecer no `dpkg -l`, tem versão e sai com `apt remove`. Adivinha os metadados — aproximado, mas resolve o problema principal.

STOW · DPKG-BUILDPACKAGE
`stow` instala cada programa sob `/usr/local/stow/` e gerencia por links: desinstalar é remover os links. A solução completa continua sendo gerar um `.deb` de verdade a partir de `apt source` + patch.

`curl -sSL … | sudo bash` **é a pior forma de instalar qualquer coisa.** Executa com privilégio total um script que você não leu, servido por um host que pode mudar de conteúdo entre a sua visita ao site e a execução — e que pode até servir conteúdo diferente para o `curl` e para o navegador. Se for o único caminho: `curl -fsSL … -o /tmp/install.sh` → `less /tmp/install.sh` (ler de verdade) → `bash /tmp/install.sh`, sem `sudo` quando possível. Lembre que **um `configure` é script shell e um `Makefile` pode conter qualquer comando**: compilar fonte não auditado na máquina que guarda credencial de cliente é escolha ruim — a VM descartável existe para isso.

RESUMO RÁPIDO

- ✓ **Compilar é o último recurso:** repositório → `pipx` → contêiner → recompilar o `.deb` com patch → do zero.
- ✓ `libfoo3` é para **executar**, `libfoo-dev` é para **compilar**: erro em `foo.h` = instale o `-dev`. `apt build-dep` resolve tudo de uma vez.
- ✓ Biblioteca em lugar fora do padrão: arquivo em `/etc/ld.so.conf.d/` + `sudo ldconfig` — **não** `LD_LIBRARY_PATH` no `.bashrc`, que confunde meses depois.
- ✓ No CMake, `-DCMAKE_BUILD_TYPE=Release` é o que liga a otimização; build fora da árvore torna a limpeza um `rm -rf build`.
- ✓ Pacote recompilado com patch precisa de `apt-mark hold`, ou o próximo `full-upgrade` o substitui.

NO KALI

A maior parte da ferramenta de segurança do GitHub é Python ou Go — e nenhuma das duas pede compilação clássica: `pipx` instala isolado e desinstala limpo; `go install` deixa um binário estático em `~/go/bin`. `make` só para C/C++ de fato. Clonar em `/opt` é convenção razoável, mas exige disciplina: um README local dizendo **de onde veio e em que commit**, e `git pull` periódico. **Adote o** `checkinstall` — cada `make install` cru é um ponto cego permanente para `dpkg -V` e `dpkg -S`. Compilar PoC é rotina de laboratório: código de exploração publicado às vezes traz erro, e ocasionalmente traz **malícia dirigida a quem compila sem ler** — leia o fonte, use VM descartável e só dentro de escopo autorizado (Lei 12.737/2012). E se o build morrer sem mensagem, olhe `dmesg -T | grep -i oom` antes do código: `-j$(nproc)` em VM de 4 GB acorda o OOM killer — caia para `-j2`.